

Employee.java

java

```
import java.util.Objects;

public class Employee {
    // Private variables
    private int id;
    private String name;
    private String department;
    private double salary;
    private String email;

    // Constructors
    public Employee() {
    }

    public Employee(int id, String name, String department, double salary, String email) {
        this.id = id;
        this.name = name;
        this.department = department;
        this.salary = salary;
        this.email = email;
    }

    // Public getters
    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public String getDepartment() {
        return department;
    }

    public double getSalary() {
```

```

        return salary;
    }

    public String getEmail() {
        return email;
    }

    // Public setters
    public void setId(int id) {
        this.id = id;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setDepartment(String department) {
        this.department = department;
    }

    public void setSalary(double salary) {
        this.salary = salary;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    // Override toString method
    @Override
    public String toString() {
        return String.format("Employee{ID: %d, Name: %-15s, Department: %-12s, Salary: $%.2f, Email: %s}",
            id, name, department, salary, email);
    }

    // Override equals method
    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;

```

```

        Employee employee = (Employee) obj;
        return id == employee.id &&
            Double.compare(employee.salary, salary) == 0 &&
            Objects.equals(name, employee.name) &&
            Objects.equals(department, employee.department) &&
            Objects.equals(email, employee.email);
    }

    // Override hashCode method
    @Override
    public int hashCode() {
        return Objects.hash(id, name, department, salary, email);
    }
}

```

EmployeeListDemo.java (Main class)

java

```

import java.util.*;

public class EmployeeListDemo {

    public static void main(String[] args) {
        // Method 1: Using ArrayList
        System.out.println("=== EMPLOYEE MANAGEMENT SYSTEM ===");
        System.out.println("\n1. Using ArrayList:");
        System.out.println("-----");
        demonstrateArrayList();

        // Method 2: Using LinkedList
        System.out.println("\n\n2. Using LinkedList:");
        System.out.println("-----");
        demonstrateLinkedList();

        // Method 3: Advanced operations
        System.out.println("\n\n3. Advanced List Operations:");
        System.out.println("-----");
        demonstrateAdvancedOperations();
    }
}

```

```

// Method to demonstrate ArrayList
public static void demonstrateArrayList() {
    // Create ArrayList to store Employee objects
    List<Employee> employeeList = new ArrayList<>();

    // Adding employees using constructor
    employeeList.add(new Employee(101, "John Doe", "IT", 75000.0,
"john.doe@company.com"));
    employeeList.add(new Employee(102, "Jane Smith", "HR", 65000.0,
"jane.smith@company.com"));
    employeeList.add(new Employee(103, "Bob Johnson", "Finance", 80000.0,
"bob.johnson@company.com"));

    // Adding employee using setter methods
    Employee emp4 = new Employee();
    emp4.setId(104);
    emp4.setName("Alice Brown");
    emp4.setDepartment("Marketing");
    emp4.setSalary(70000.0);
    emp4.setEmail("alice.brown@company.com");
    employeeList.add(emp4);

    // Adding another employee
    employeeList.add(new Employee(105, "Charlie Wilson", "IT", 72000.0,
"charlie.wilson@company.com"));

    // Print all employees
    printEmployees(employeeList, "ArrayList");

    // Additional ArrayList specific operations
    System.out.println("\n--- ArrayList Specific Info ---");
    System.out.println("Size of ArrayList: " + employeeList.size());
    System.out.println("Is ArrayList empty? " + employeeList.isEmpty());
    System.out.println("Index of Bob Johnson: " + getEmployeeIndex(employeeList, "Bob
Johnson"));
}

// Method to demonstrate LinkedList
public static void demonstrateLinkedList() {
    // Create LinkedList to store Employee objects

```

```

List<Employee> employeeLinkedList = new LinkedList<>();

// Adding employees
employeeLinkedList.add(new Employee(201, "David Miller", "Sales", 68000.0,
"david.miller@company.com"));
employeeLinkedList.add(new Employee(202, "Emma Davis", "IT", 78000.0,
"emma.davis@company.com"));
employeeLinkedList.add(new Employee(203, "Frank Thomas", "Operations", 72000.0,
"frank.thomas@company.com"));

// Adding employee at specific position (LinkedList feature)
((LinkedList<Employee>) employeeLinkedList).addFirst(
    new Employee(200, "Zoe Adams", "Executive", 95000.0, "zoe.adams@company.com")
);

// Adding at the end
((LinkedList<Employee>) employeeLinkedList).addLast(
    new Employee(204, "Grace Lee", "HR", 67000.0, "grace.lee@company.com")
);

// Print all employees
printEmployees(employeeLinkedList, "LinkedList");

// Additional LinkedList specific operations
System.out.println("\n--- LinkedList Specific Info ---");
System.out.println("First Employee: " + ((LinkedList<Employee>)
employeeLinkedList).getFirst());
System.out.println("Last Employee: " + ((LinkedList<Employee>)
employeeLinkedList).getLast());
System.out.println("Total Employees: " + employeeLinkedList.size());
}

// Method to demonstrate advanced operations
public static void demonstrateAdvancedOperations() {
    List<Employee> employees = new ArrayList<>();

    // Adding multiple employees
    employees.add(new Employee(301, "Henry Clark", "IT", 82000.0,
"henry.clark@company.com"));
    employees.add(new Employee(302, "Irene Baker", "Finance", 74000.0,
"irene.baker@company.com"));

```

```

        employees.add(new Employee(303, "Jack White", "Marketing", 69000.0,
"jack.white@company.com"));
        employees.add(new Employee(304, "Karen Young", "HR", 71000.0,
"karen.young@company.com"));

// Print original list
System.out.println("Original List:");
printEmployees(employees, "Original");

// 1. Iterating using different methods
System.out.println("\n--- Iteration Methods ---");

// Method 1: Enhanced for loop
System.out.println("Using Enhanced For Loop:");
for (Employee emp : employees) {
    System.out.println("  " + emp.getName() + " - " + emp.getDepartment());
}

// Method 2: ForEach with lambda (Java 8+)
System.out.println("\nUsing forEach with Lambda:");
employees.forEach(emp -> System.out.println("  " + emp.getName() + " - $" +
emp.getSalary()));

// Method 3: Iterator
System.out.println("\nUsing Iterator:");
Iterator<Employee> iterator = employees.iterator();
while (iterator.hasNext()) {
    Employee emp = iterator.next();
    System.out.println("  " + emp);
}

// 2. Searching and filtering
System.out.println("\n--- Searching Operations ---");

// Find employee by ID
int searchId = 302;
Employee found = findEmployeeById(employees, searchId);
if (found != null) {
    System.out.println("Found employee with ID " + searchId + ": " + found.getName());
}

```

```

// Find employees in IT department
System.out.println("\nEmployees in IT Department:");
employees.stream()
    .filter(emp -> "IT".equals(emp.getDepartment()))
    .forEach(emp -> System.out.println("  " + emp.getName()));

// 3. Sorting operations
System.out.println("\n--- Sorting Operations ---");

// Sort by name
List<Employee> sortedByName = new ArrayList<>(employees);
sortedByName.sort(Comparator.comparing(Employee::getName));
System.out.println("Sorted by Name:");
sortedByName.forEach(emp -> System.out.println("  " + emp.getName() + " - " +
emp.getDepartment()));

// Sort by salary (descending)
List<Employee> sortedBySalary = new ArrayList<>(employees);
sortedBySalary.sort(Comparator.comparing(Employee::getSalary).reversed());
System.out.println("\nSorted by Salary (Highest first):");
sortedBySalary.forEach(emp -> System.out.printf("  %s - $%.2f%n", emp.getName(),
emp.getSalary()));

// 4. Update operations
System.out.println("\n--- Update Operations ---");
updateEmployeeSalary(employees, 303, 75000.0);
System.out.println("After updating salary of employee 303:");
printEmployees(employees, "Updated");

// 5. Remove operations
System.out.println("\n--- Remove Operations ---");
removeEmployeeById(employees, 304);
System.out.println("After removing employee 304:");
printEmployees(employees, "After Removal");
}

// Helper method to print all employees
public static void printEmployees(List<Employee> employees, String listType) {
    System.out.println("\n" + listType + " Employee List:");
    System.out.println("=".repeat(60));
    if (employees.isEmpty()) {

```

```

        System.out.println("No employees found.");
    } else {
        for (Employee emp : employees) {
            System.out.println(emp);
        }
        System.out.println("=".repeat(60));
        System.out.println("Total Employees: " + employees.size());
    }
}

// Helper method to find employee by ID
public static Employee findEmployeeById(List<Employee> employees, int id) {
    for (Employee emp : employees) {
        if (emp.getId() == id) {
            return emp;
        }
    }
    return null;
}

// Helper method to get index of employee by name
public static int getEmployeeIndex(List<Employee> employees, String name) {
    for (int i = 0; i < employees.size(); i++) {
        if (employees.get(i).getName().equals(name)) {
            return i;
        }
    }
    return -1;
}

// Helper method to update employee salary
public static void updateEmployeeSalary(List<Employee> employees, int id, double
newSalary) {
    Employee emp = findEmployeeById(employees, id);
    if (emp != null) {
        emp.setSalary(newSalary);
        System.out.println("Updated salary for " + emp.getName() + " to $" + newSalary);
    } else {
        System.out.println("Employee with ID " + id + " not found.");
    }
}
}

```

```
// Helper method to remove employee by ID
public static void removeEmployeeById(List<Employee> employees, int id) {
    Iterator<Employee> iterator = employees.iterator();
    while (iterator.hasNext()) {
        Employee emp = iterator.next();
        if (emp.getId() == id) {
            iterator.remove();
            System.out.println("Removed employee: " + emp.getName());
            return;
        }
    }
    System.out.println("Employee with ID " + id + " not found.");
}
}
```